



АКАДЕМИЯ
КОДЕБАЙ

ПРОФЕССИЯ ПЕНТЕСТЕР

Всем доброго времени суток!

В этом разделе мы сосредоточимся на выявлении и эксплуатации распространенных уязвимостей веб-приложений. Современные платформы разработки и хостинговые решения упростили процесс создания и развертывания веб-приложений. Однако эти приложения обычно представляют собой большую поверхность для атак из-за многочисленных зависимостей и небезопасных конфигураций сервера. Веб-приложения могут быть написаны на различных языках программирования и фреймворках, каждые из которых могут создавать определенные типы уязвимостей. Хотя сложность уязвимостей и атак различна, я продемонстрирую использование нескольких распространенных уязвимостей веб-приложений, перечисленных в списке [OWASP Top 10](#). Эти векторы атак послужат в качестве базовых строительных блоков, используемых для построения более сложных атак.

Методология оценки веб-приложений

Прежде чем приступить к обсуждению перечисления и эксплуатации, мы поговорим об основной методологии тестирования веб-приложений на проникновение. В качестве первого шага мы должны собрать информацию о приложении. Что делает приложение? На каком языке оно написано? На каком серверном программном обеспечении работает приложение? Ответы на эти и другие основные вопросы помогут нам найти первый вектор атаки. Как и во многих других дисциплинах тестирования на проникновение, целью каждой попытки атаки или эксплойта является увеличение наших полномочий в приложении или переход к другому приложению или цели. Каждый успешный эксплойт на этом пути может предоставить доступ к новым функциональным возможностям или компонентам приложения. Нам может понадобиться успешно выполнить несколько эксплойтов, чтобы перейти от



неаутентифицированной учетной записи пользователя к любому виду оболочки в системе. Перечисление новых функциональных возможностей важно на каждом шаге пути, особенно потому, что атаки, которые ранее не удались, могут быть успешными в новом контексте. Как тестеры проникновения, мы должны продолжать перечислять и адаптироваться, пока не исчерпаем все пути атаки или не скомпрометируем систему.

Энумерация веб-приложений

Важно определить компоненты, из которых состоит веб-приложение, прежде чем пытаться использовать его вслепую. Многие уязвимости веб-приложений не зависят от технологии. Однако некоторые эксплойты и полезные нагрузки должны быть разработаны с учетом технологической основы приложения, например, программного обеспечения базы данных или операционной системы. Перед началом атак на веб-приложение следует попытаться обнаружить используемый технологический стек, который обычно состоит из следующих компонентов:

1. Языки программирования и фреймворки
2. Программное обеспечение веб-сервера
3. Программное обеспечение базы данных
4. Серверная операционная система

Существует несколько методов, которые мы можем использовать для сбора этой информации непосредственно из браузера. Большинство современных браузеров включают инструменты разработчика, которые могут помочь в процессе перечисления. Мы сфокусируемся на Firefox, поскольку он является браузером по умолчанию в Kali Linux.

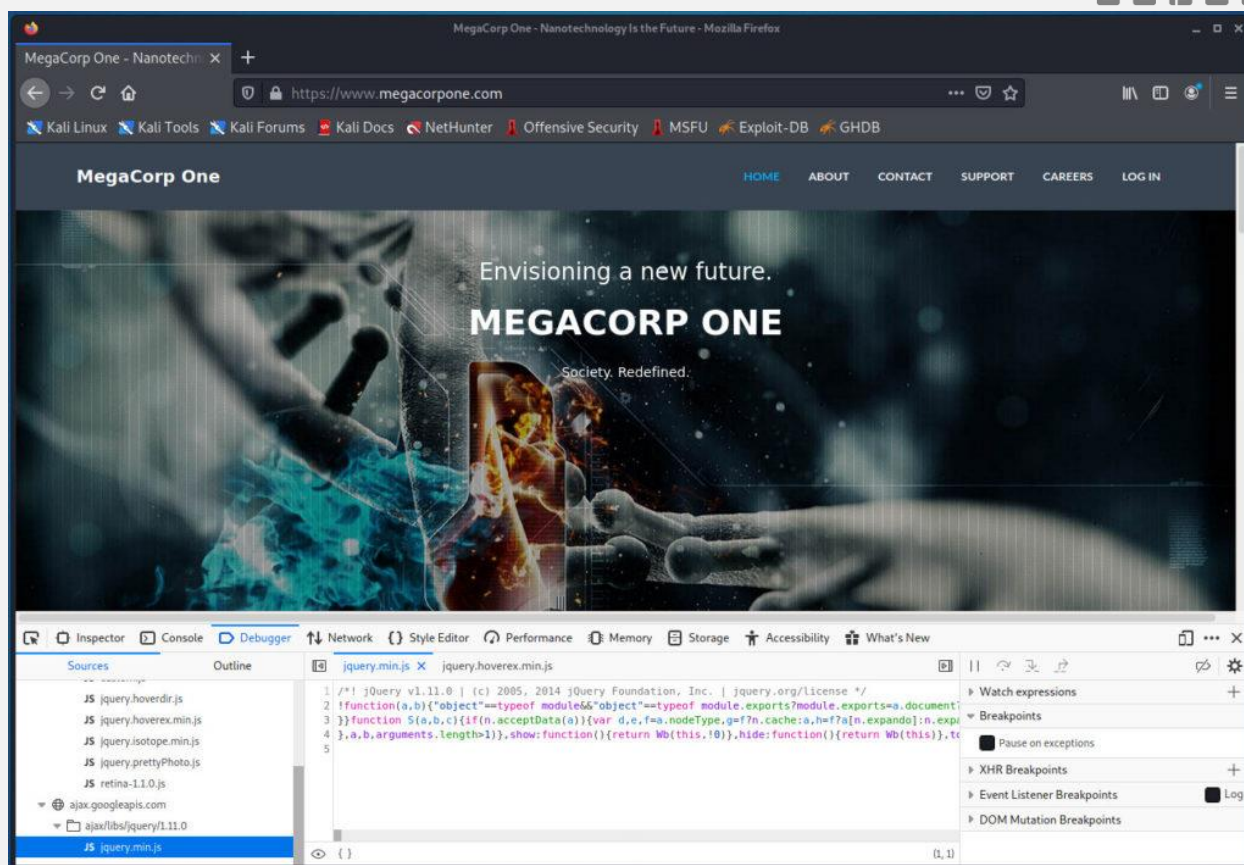


Проверка URL-адресов

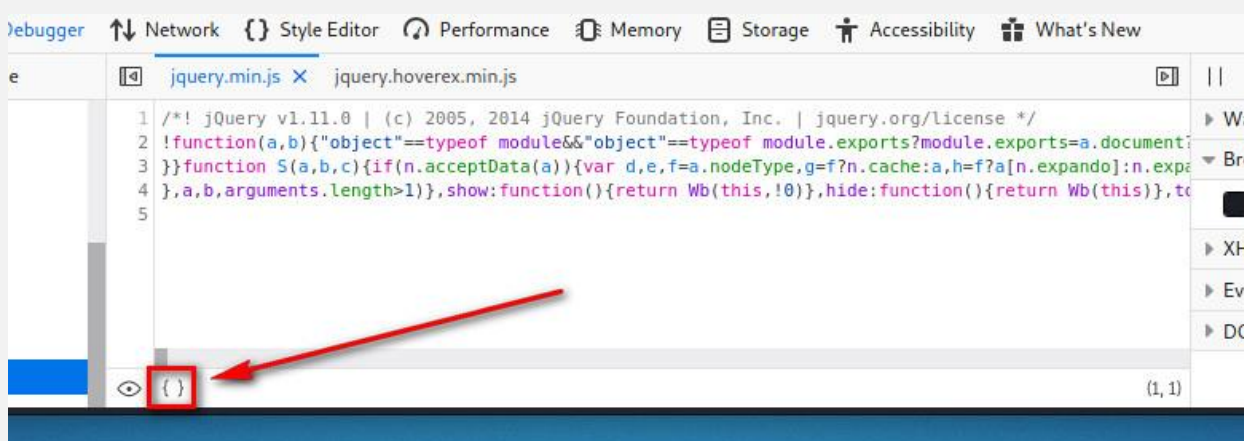
Расширения файлов, которые иногда являются частью URL, могут показать язык программирования, на котором написано WEB-приложение. Некоторые из них, например .php, просты, но другие расширения более загадочны и зависят от используемых фреймворков. Например, веб-приложение на базе Java может использовать .jsp, .do или .html. Однако расширения файлов на веб-страницах становятся все менее распространенными, поскольку многие языки и фреймворки теперь поддерживают концепцию "маршрутов", которые позволяют разработчикам сопоставить [URI](#) с секцией кода. Приложения, использующие такие "маршруты", используют логику для определения того, какой контент должен возвращаться пользователю.

Проверка содержимого страницы

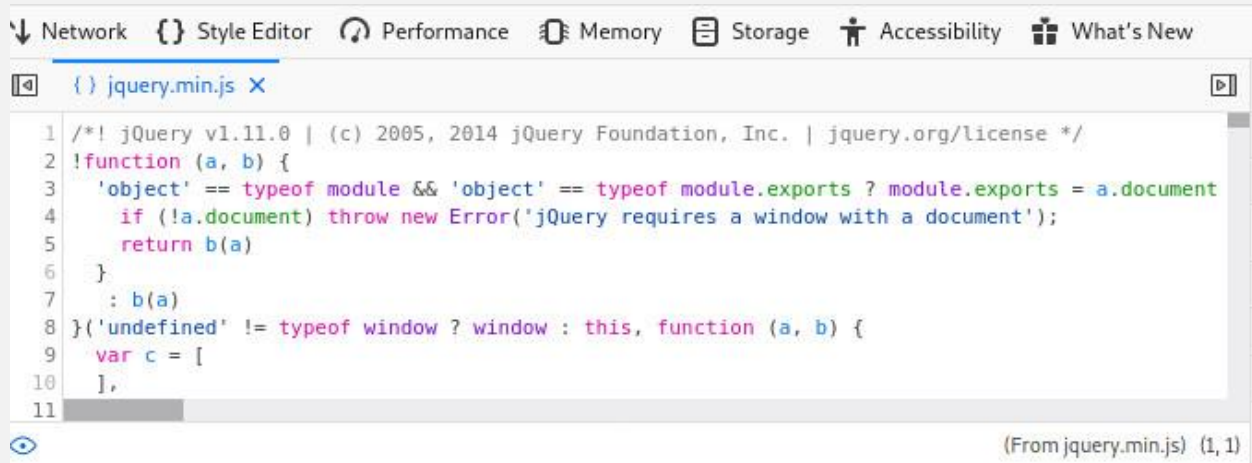
Хотя проверка URL-адресов может дать некоторые подсказки о целевом веб-приложении, большинство подсказок можно найти в исходном тексте веб-страницы. Инструмент Firefox Debugger (находится в меню Web Developer или можно нажать сочетание клавиш Ctrl+Shift+K) отображает ресурсы и содержимое страницы, которое варьируется в зависимости от веб-приложения. Инструмент Debugger может отображать JavaScript фреймворки, скрытые поля ввода, комментарии, элементы управления на стороне клиента, и многое другое. Чтобы продемонстрировать это, мы можем открыть отладчик (Debugger) во время просмотра сайта <https://www.megacorpone.com>:



Мы видим, что приложение, запущенное на www.megacorpone.com, использует [jQuery](#) версии 1.11.0, распространенную библиотеку JavaScript. В данном случае разработчик [минифицировал](#) код, сделав его более компактным и экономящим ресурсы, но делающим его несколько трудным для чтения. К счастью, мы можем "приукрасить" код в Firefox, нажав на кнопку "Pretty print source" с двойными фигурными скобками:



После щелчка по значку Firefox отобразит код в формате, удобном для чтения и исследования:

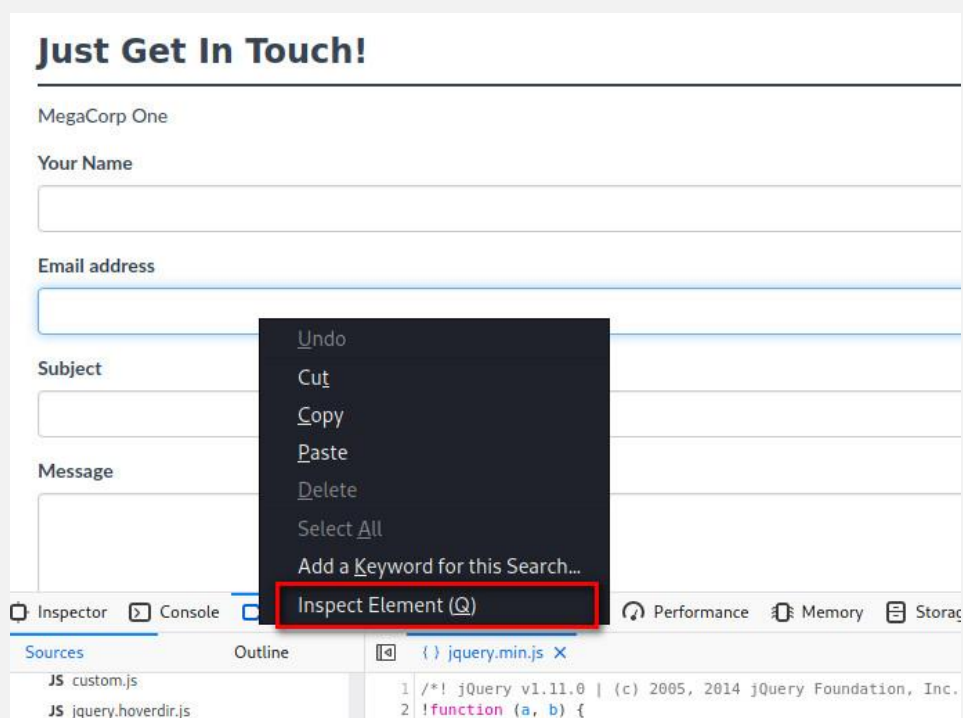


```

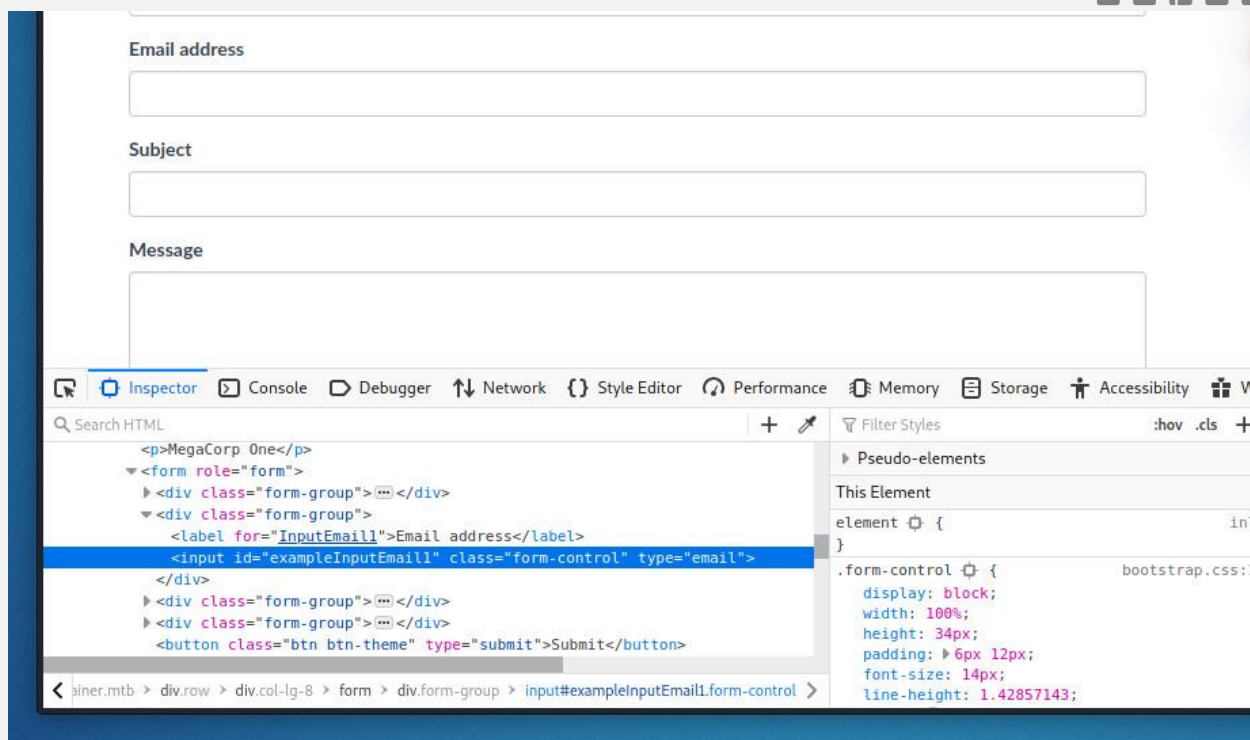
1  /*! jQuery v1.11.0 | (c) 2005, 2014 jQuery Foundation, Inc. | jquery.org/license */
2  !function (a, b) {
3    'object' == typeof module && 'object' == typeof module.exports ? module.exports = a.document
4      if (!a.document) throw new Error('jQuery requires a window with a document');
5      return b(a)
6    }
7    : b(a)
8  }('undefined' != typeof window ? window : this, function (a, b) {
9    var c = [
10
11

```

Мы также можем использовать инструмент Inspector для углубления в конкретное содержимое страницы. Давайте воспользуемся инспектором, чтобы изучить элемент ввода электронной почты на странице "Контакты", щелкнув правой кнопкой мыши на поле адреса электронной почты и выбрав пункт Inspect Element:



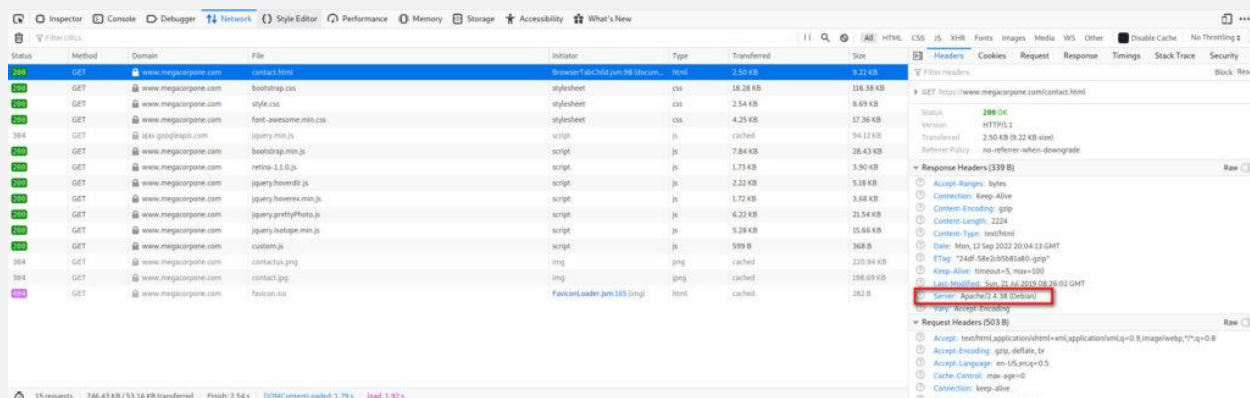
Это действие откроет инструмент Inspector и выделит HTML для элемента, на котором мы щелкнули правой кнопкой мыши:



Этот инструмент особенно полезен для быстрого поиска скрытых полей формы в HTML коде.

Просмотр заголовков ответов сервера

Мы также можем искать дополнительную информацию в ответах сервера. Существует два типа инструментов, которые можно использовать для выполнения этой задачи. Первый тип инструментов — это прокси, который перехватывает запросы и ответы между клиентом и веб-сервером. Мы рассмотрим прокси позже в этом разделе (когда будем работать с Burp Suite), но сначала мы изучим инструмент Network, запускаемый из меню Web Developer, для просмотра HTTP-запросов и ответов. Этот инструмент показывает сетевую активность, которая происходит после его запуска, поэтому мы должны обновить страницу, чтобы увидеть трафик.

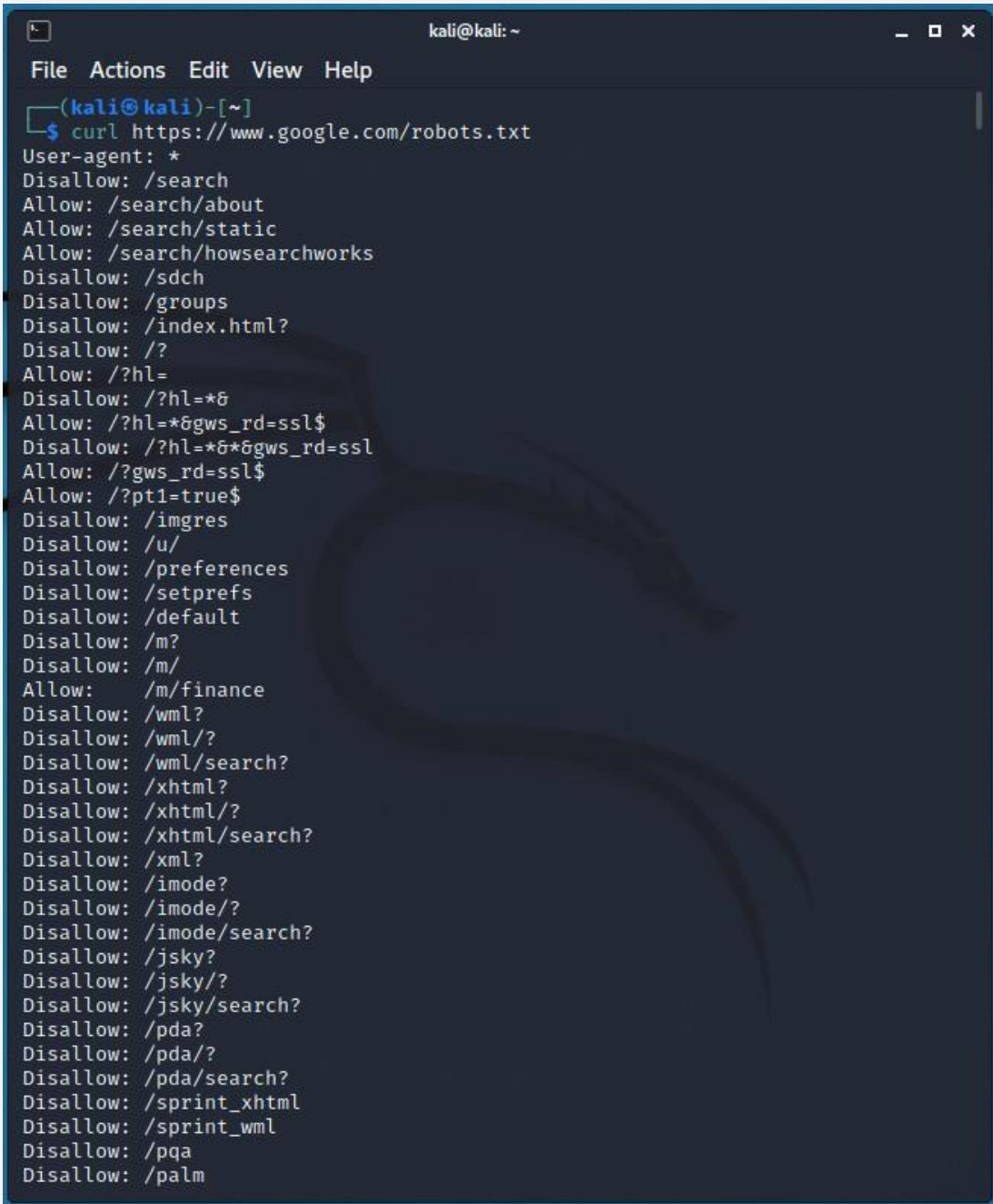


Заголовок "Server", часто показывает, по крайней мере, имя программного обеспечения веб-сервера. Во многих конфигурациях по умолчанию он также показывает номер версии. Заголовки, начинающиеся с "X-", являются нестандартными HTTP-заголовками. Их названия или значения часто раскрывают дополнительную информацию о технологическом стеке, используемом приложением. Некоторые нестандартные заголовки включают "X-Powered-By", "x-amz-cf-id" и "X-AspNet-Version". Дальнейшее изучение этих названий может выявить дополнительную информацию. Например, заголовок "x-amz-cf-id", указывает на то, что приложение использует [Amazon CloudFront](#).

Исследование Sitemaps

Веб-приложения могут использовать файлы Sitemap (чаще всего так и происходит), чтобы помочь ботам поисковых систем проиндексировать их сайты. Эти файлы также содержат директивы о том, какие URL-адреса не следует просматривать. Как правило, это конфиденциальные страницы или административные консоли - именно те страницы, которые нас интересуют. Два наиболее распространенных имени файлов Sitemap - robots.txt и sitemap.xml. Например, мы можем получить файл robots.txt с сайта www.google.com с помощью команды curl:

```
kali@kali:~$ curl https://www.google.com/robots.txt
```



```
kali@kali: ~  
File Actions Edit View Help  
(kali@kali)-[~]  
$ curl https://www.google.com/robots.txt  
User-agent: *  
Disallow: /search  
Allow: /search/about  
Allow: /search/static  
Allow: /search/howsearchworks  
Disallow: /sdch  
Disallow: /groups  
Disallow: /index.html?  
Disallow: /?  
Allow: /?hl=  
Disallow: /?hl=*  
Allow: /?hl=*gws_rd=ssl$  
Disallow: /?hl=*gws_rd=ssl$  
Allow: /?gws_rd=ssl$  
Allow: /?pt1=true$  
Disallow: /imgres  
Disallow: /u/  
Disallow: /preferences  
Disallow: /setprefs  
Disallow: /default  
Disallow: /m?  
Disallow: /m/  
Allow: /m/finance  
Disallow: /wml?  
Disallow: /wml/?  
Disallow: /wml/search?  
Disallow: /xhtml?  
Disallow: /xhtml/?  
Disallow: /xhtml/search?  
Disallow: /xml?  
Disallow: /imode?  
Disallow: /imode/?  
Disallow: /imode/search?  
Disallow: /jsky?  
Disallow: /jsky/?  
Disallow: /jsky/search?  
Disallow: /pda?  
Disallow: /pda/?  
Disallow: /pda/search?  
Disallow: /sprint_xhtml  
Disallow: /sprint_wml  
Disallow: /pqa  
Disallow: /palm
```

Allow и Disallow - это директивы для [веб-краулеров](#), указывающие на страницы или каталоги, к которым "вежливые" веб-краулеры могут обращаться или не обращаться соответственно. Хотя перечисленные страницы и каталоги в большинстве случаев могут быть неинтересными, а некоторые даже недействительными, файлы Sitemap не следует игнорировать, поскольку они могут содержать подсказки о схеме сайта или другую интересную информацию.

Нахождение консолей администрирования

Веб-серверы часто поставляются с веб-приложениями для удаленного администрирования, или консолями, которые доступны по определенному URL и часто прослушивают определенный TCP-порт. Двумя распространенными примерами являются приложения manager для [Tomcat](#) и phpMyAdmin для [MySQL](#) расположенные по адресам /manager/html и /phpmyadmin соответственно. Хотя эти консоли могут быть ограничены локальным доступом или размещаться на пользовательских TCP-портах, мы часто можем найти их открытыми для внешнего доступа. В любом случае мы должны проверять расположение этих консолей по умолчанию, указанное в документации к программному обеспечению сервера приложений. В одном из уроков мы также изучим инструменты ("дирсерчеры"), которые можно использовать для автоматизации поиска этих консолей.

Команды, используемые в уроке:

```
kali@kali:~$ curl https://www.google.com/robots.txt
```